

Министерство науки и высшего образования РФ
ФГБОУ ВО «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и
управления»

ОТЧЁТ
по проектной деятельности
студента Шульгиной Валерии Игоревны группы ПИН-221

Пояснительная записка
Шифр работы От-2068998-43-ПИН-221-51 ПЗ
Специальность 09.03.04

Руководитель, доцент к.т.н

Е.О. Викулов

Студент

В.И. Шульгина

Омск 2025

РЕФЕРАТ

Отчет 30 с., 1 ч., 11 рис.

ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ, АВТОМАТИЗАЦИЯ
ОБРАЗОВАНИЯ, ПРОГРАММНАЯ ИНЖЕНЕРИЯ, CRM, LMS, WEB-
ПРИЛОЖЕНИЕ

Объект исследования – автоматизированные процессы школы программирования для детей.

Цель работы – проектирование автоматизированной системы управления образовательным процессом, финансами, коммуникациями и цифровым портфолио учеников детской школы программирования.

В процессе работы были проанализированы отечественные и зарубежные аналоги, сформулированы функциональные и нефункциональные требования, выбраны методологии и стандарты программной инженерии, а также разработан комплект архитектурных моделей.

В результате выполнения работы сформирована архитектура автоматизированной информационной системы школы программирования, включающая модули пользователей и ролей, учебного процесса, финансов и уведомлений, портфолио и проектной ленты.

Содержание

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	4
ПЕРЕЧЕНЬ ТЕРМИНОВ И ОПРЕДЕЛЕНИЙ	5
ВВЕДЕНИЕ	6
Глава 1. Анализ решений и методологические основы проектирования	8
1.1 Анализ аналогичных систем и отечественного/зарубежного опыта	8
1.2 Формализация требований к автоматизированной системе	12
1.2.1 Функциональные требования	12
1.2.2 Нефункциональные требования	13
1.3 Методологии и стандарты программной инженерии для разработки автоматизированных систем управления ОДО	14
1.3.1 Методологии жизненного цикла разработки ПО	15
1.3.2 Архитектурные паттерны и подходы	15
1.3.3 Стандарты в области качества и разработки программного обеспечения	16
1.3.4 Стандарты и нормативная база в сфере защиты данных и информационной безопасности	17
1.3.5 Стандарты документирования	18
1.4 Моделирование архитектуры: UML, IDEF0 и CASE-среды	18
Глава 2. Проектирование и реализация автоматизированной системы для школы программирования	27
2.1 Архитектурное решение	27
2.2 Ключевые функциональные модули	27
2.3 Выбор программных решений	27
2.3.1 CASE – средства	27
2.3.2 Стек серверной части	28
2.3.3 Стек клиентской части	28
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете по курсовому проекту применяют следующие сокращения и обозначения:

API – программный интерфейс прикладного программирования

LMS – система управления обучением

UML – унифицированный язык моделирования

REST – архитектурный стиль взаимодействия распределенных приложений

PDF – формат электронных документов

SQL – язык структурированных запросов к реляционным базам данных

ФЗ-152 – Федеральный закон Российской Федерации «О персональных данных»

ПЕРЕЧЕНЬ ТЕРМИНОВ И ОПРЕДЕЛЕНИЙ

В настоящем отчете по курсовому проекту применяют следующие термины и определения:

Автоматизированная система управления образовательным процессом – программная система, обеспечивающая поддержку учебных, административных и коммуникационных процессов школы.

Смарт-отработка пропуска – механизм автоматического предложения и бронирования слотов отработки пропущенного занятия с подтверждением родителем.

Антиотток-аналитика – набор алгоритмов анализа поведения учеников для выявления рисков прекращения обучения.

Цифровое портфолио ученика – структурированная подборка выполненных учеником проектов с возможностью формирования PDF-отчета.

Проектная лента – регулярная публикация учебных работ учеников с возможностью голосования и определения лучшей работы недели.

ВВЕДЕНИЕ

Разработка автоматизированной системы управления для предприятия дополнительного образования является актуальной задачей на стыке программной инженерии и цифровой трансформации образовательной отрасли. В данной работе рассматривается система, ориентированная на автоматизацию бизнес-процессов детской школы программирования KiberOne — одной из крупнейших франшиз IT-образования для детей в России. Особенностью данного класса организаций является специфика, не учтённая в универсальных CRM и LMS-системах: цикличность абонементной оплаты, необходимость организации отработок пропущенных занятий, двусторонняя коммуникация с родителями и трекинг образовательных компетенций в долгосрочной перспективе.

Разрабатываемая система решает комплекс задач, недостаточно покрытых существующими решениями общего назначения (Битрикс24, AmoCRM, GetCourse, 1С:Образование). В их числе: автоматическая генерация цифрового портфолио ученика по накопленным проектам с выгрузкой в PDF, выявление учеников в зоне риска отказа от обучения на основе поведенческих паттернов (пропуски, снижение активности на занятиях, система подбора слотов для отработки пропущенных занятий. Дополнительно система включает личный кабинет ученика с элементами геймификации — прокачкой персонажа по мере освоения курсов — и социальную ленту проектов с механизмом голосования за лучшие работы, что формирует образовательное сообщество внутри школы.

Актуальность разработки обусловлена высокой конкуренцией на рынке детского дополнительного образования и растущими ожиданиями родителей: они готовы платить за обучение лишь при наличии осязаемых, измеримых результатов. Автоматически сформированное портфолио с проектами ребёнка становится документально подтверждённым результатом обучения. Аналитика оттока, в свою очередь, позволяет менеджерам школы быстро реагировать, сокращая потери клиентской базы.

Практическая значимость работы состоит в снижении операционной нагрузки на менеджеров и преподавателей, повышении прозрачности образовательного процесса для родителей и формировании конкурентного преимущества школы за счёт уникальных функций, отсутствующих в массовых аналогах. Исследуемая задача непосредственно относится к области программной инженерии, так как требует проектирования архитектуры веб-приложения, разработки алгоритмов аналитики, работы с реляционными базами данных, генерации документов и интеграции уведомлений через мессенджеры.

Цель исследования — спроектировать и реализовать автоматизированную информационную систему, учитывающую специфику детской школы программирования и закрывающую пробелы универсальных решений: автоматизацию формирования портфолио учеников, управление удержанием клиентов и планирование отработок, — тем самым минимизируя ручной труд сотрудников и повышая ценность образовательного продукта для конечного потребителя.

Таким образом, разрабатываемая система необходима для решения актуальных задач цифровизации предприятий сферы дополнительного образования, что полностью соответствует современным требованиям к программной инженерии и обоснованно представляется к защите по данному направлению.

Глава 1. Анализ решений и методологические основы проектирования

1.1 Анализ аналогичных систем и отечественного/зарубежного опыта

Данный раздел посвящён исследованию существующих решений в области автоматизации управления предприятиями дополнительного образования с акцентом на специфику детской школы программирования KiberOne. Цель анализа — выявить применимые практики и ограничения типовых платформ CRM/LMS для последующего использования при проектировании разрабатываемой автоматизированной системы.

Анализ отечественного опыта

В качестве основных отечественных аналогов рассмотрены **Битрикс24**, **1С:Образование**, **АльфаCRM**, а также практики использования локальных решений в центрах дополнительного образования.

Битрикс24 обеспечивает развитый CRM-контур: ведение карточек клиентов, воронки продаж, постановку задач менеджерам, базовые уведомления и отчётность. Платформа удобна для сопровождения первичных продаж и повторных контактов с родителями, что частично закрывает потребности по управлению клиентской базой.

При этом для образовательной деятельности требуются дополнительные доработки: штатно отсутствует полноценный академический журнал с привязкой к компетенциям, нет специализированной логики абонементной модели обучения и инструментов «смарт-отработки» пропусков по уровню ученика и расписанию группы.

1С:Образование имеет более выраженную учебную направленность: учёт контингента, элементы учебного процесса и административной отчётности. Решение подходит для формализованных сценариев документооборота и может использоваться в связке с финансовыми модулями 1С.

Однако для частных школ программирования возникает недостаточная гибкость пользовательских сценариев: функционал часто ориентирован на классические учреждения, а не на продуктовую модель дополнительного IT-образования с геймификацией, проектной лентой и автоматическим формированием маркетингового портфолио учеников.

АльфаCRM — это отраслевая CRM-система, разработанная специально для центров дополнительного образования: языковых школ, детских студий, спортивных секций и IT-кружков. Среди рассматриваемых отечественных аналогов она является наиболее близкой к целевой предметной области по охвату функциональности.

Система поддерживает: ведение клиентской базы и карточек учеников, управление учебными группами и расписанием, журнал посещаемости, абонементную модель оплаты с автоматическими напоминаниями, воронку продаж с этапами и задачами для менеджеров, а также базовую отчетность по финансам. Реализована интеграция с мессенджерами для отправки уведомлений.

Вместе с тем АльфаCRM обладает рядом ограничений. В системе отсутствуют: механизмы автоматической генерации цифрового портфолио ученика и его экспорта в PDF, аналитика рисков оттока на основе поведенческих паттернов, интеллектуальный подбор слотов для отработки пропусков, детский личный кабинет с геймификацией и прокачкой персонажа, а также социальная лента проектов с голосованием. Таким образом, платформа закрывает операционные задачи администрирования, но не формирует дополнительной ценности для конечных потребителей — детей и родителей.

Преимущества отечественных решений: соответствие российской практике документооборота и локальным требованиям; удобство внедрения в существующие процессы продаж и администрирования; развитая экосистема интеграторов на российском рынке.

Недостатки: фрагментарное покрытие образовательных процессов в одной системе; высокая доля ручных операций по расписанию, отработкам и сопровождению прогресса; отсутствие «из коробки» специализированных функций для школы программирования: антиотток-аналитики, PDF-портфолио, геймификации, проектной ленты с голосованием.

Анализ зарубежного опыта:

В качестве зарубежных ориентиров рассмотрены **HubSpot CRM, Salesforce Education Cloud, Moodle и Canvas LMS.**

Зарубежные CRM-платформы (**HubSpot, Salesforce Education Cloud**) демонстрируют зрелые механизмы работы с клиентским циклом: сегментация, сквозная аналитика, автоматизированные коммуникации, события и триггеры удержания. Особенно ценной является развитая аналитика поведения пользователей и сценарии автоматических реакций на риски оттока.

Вместе с тем их внедрение в России часто требует локализации процессов, юридической адаптации и интеграции с локальными каналами коммуникации и платежами.

Moodle и Canvas предоставляют широкие возможности учебного администрирования: курсы, задания, оценивание, отчёты по прогрессу. Они подходят для построения академического трекинга и цифрового следа ученика.

Ограничение для бизнес-модели дополнительного образования состоит в том, что CRM-функции, абонементная логика и операционные процессы филиальной сети обычно реализуются через дополнительные модули или внешние интеграции.

Преимущества зарубежных решений: высокий уровень зрелости API и аналитических инструментов; развитые инструменты по удержанию клиентов; качественные UX-подходы к личным кабинетам и образовательной мотивации.

Ограничения: стоимость владения и сложность внедрения; необходимость адаптации под российские нормативные и организационные условия; не всегда прямое соответствие локальным сценариям взаимодействия школы и родителей.

Сопоставление рассмотренных аналогов с требованиями проектируемой системы позволяет выявить следующие закономерности.

Ведение клиентской базы, воронки продаж и автоматические задачи для менеджеров реализованы в Битрикс24 и amoCRM на достаточном уровне. Однако обе платформы не обеспечивают привязку истории взаимодействий к образовательному прогрессу конкретного ребёнка.

Управление расписанием, учебными группами, журналом посещаемости и абонементной оплатой наиболее полно реализовано в АльфаCRM, что делает её ближайшим операционным аналогом. Тем не менее журнал в ней не связан с компетенциями и навыками учеников, а отработки пропусков требуют ручного управления.

Ведение учебного контента, назначение заданий и отслеживание прогресса по курсам реализовано в Moodle и Canvas. При этом данные платформы не имеют встроенной CRM-логики, финансового управления и механики работы с родителями как клиентами.

Полностью отсутствуют во всех рассмотренных системах следующие возможности, принципиально важные для школы программирования:

- автоматическое формирование цифрового портфолио ученика по накопленным проектам с выгрузкой в PDF;
- аналитическое выявление учеников в зоне риска оттока на основе поведенческих паттернов (повторные пропуски, спад активности на занятиях, нарушение платёжной дисциплины) и формирование оперативного списка для менеджеров;

- интеллектуальный подбор слотов для отработки пропущенных занятий с учётом уровня ученика и расписания групп, с подтверждением от родителя в один клик;
- детский личный кабинет с механикой геймификации — прокачкой персонажа за посещения, выполнение заданий и добавление проектов;
- социальная лента проектов с фильтрами, механизмом голосования и автоматическим определением лучших работ.

Таким образом, прямого готового решения, полностью закрывающего все потребности школы программирования, на рынке не выявлено. Наиболее рациональным является проектирование специализированной системы, объединяющей CRM-, LMS- и аналитический функционалы в единой платформе.

1.2 Формализация требований к автоматизированной системе

Формализация требований — ключевой этап проектирования, позволяющий определить, какими свойствами, функциями и характеристиками должна обладать разрабатываемая автоматизированная система для предприятия дополнительного образования. Все требования делятся на функциональные и нефункциональные.

1.2.1 Функциональные требования

Для реализации проектируемой системы важно сформулировать перечень функциональных требований, отражающих ключевые задачи, которые должна выполнять система. В данном разделе приводится описание основных функций, обеспечивающих работоспособность, удобство и эффективность использования программного продукта в рамках поставленных целей и бизнес-процессов компании-заказчика.

Функциональные требования определяют ожидаемое поведение системы, набор реализуемых сценариев, а также взаимодействие пользователя

с программным обеспечением и внешними компонентами автоматизированной среды. Был сформирован список функциональных требований к системе:

- **Управление пользователями и ролями** (отдельные аккаунты для разных категорий пользователей, привязка детей к родителям);
- **CRM и клиентская база** (карточки родителей и детей, история взаимодействий. Воронка продаж, сегментация, автоматические задачи менеджерам);
- **Курсы и программы** (справочник курсов, уровней, навыков);
- **Расписание и журнал** (формирование групп и расписания. Учёт посещаемости, ДЗ, оценок, комментариев);
- **Финансы** (Абонементы, счета, напоминания об оплате. Отчёты по платежам);
- **Уведомления** (Telegram, push. Автоматические напоминания)
- **Автоматическое портфолио ученика** (Добавление проекта в 1 клик (ссылка + описание + скрин). Генерация PDF с обложкой KiberOne, навыками, QR-кодом. Анонимный сбор лучших работ для маркетинга.)
- **Детский кабинет + геймификация** (Прокачка персонажа/аватара за активность (проекты, ДЗ, посещения). Просмотр прогресса, портфолио, достижений);
- **Лента проектов и голосование** (Общая лента проектов с фильтрами. Голосование, «Проект недели/месяца»);
- **Антиотток** (Авто-выявление риска (пропуски 2+, спад активности на занятиях). Список «позвонить сегодня» для менеджеров)
- **Смарт-отработка пропусков** (Авто-предложение 2–3 слотов по уровню и расписанию. Уведомление родителю → подтверждение в 1 клик → запись);
- **Отчёты и аналитика** (Посещаемость, успеваемость, удержание, финансы. Дашборды для каждого филиала);
- **Администрирование** (Управление филиалами, шаблонами, настройками. Журнал действий, резервное копирование).

1.2.2 Нефункциональные требования

В данном разделе рассматриваются нефункциональные требования, которые определяют качества и характеристики системы, не связанные напрямую с её функциональностью. Описание этих требований необходимо

для обеспечения надёжности, производительности, удобства использования, безопасности и других важных аспектов функционирования программного продукта в реальных условиях эксплуатации.

Нефункциональные требования служат ориентиром для оценки уровня качества, устойчивости и соответствия системы ожидаемым стандартам предприятия и профессиональной отрасли. Был сформирован список нефункциональных требований к системе:

- Удобство и простота работы для пользователей: интуитивно понятный интерфейс для менеджера/ребенка/родителя;
- Производительность: одновременная работа ≥ 500 активных пользователей;
- Надёжность: гарантируется отсутствие потери данных;
- Расширяемость: горизонтальное масштабирование для роста сети филиалов;
- Совместимость: поддержка актуальных версий браузеров (Chrome, Safari, Firefox, Edge);
- Безопасность: соответствие ФЗ-152 (персональные данные детей и родителей);
- Документирование: наличие подробной технической и пользовательской документации, поддержка журналирования операций.

1.3 Методологии и стандарты программной инженерии для разработки автоматизированных систем управления ОДО

Проектирование и разработка автоматизированной информационной системы для предприятия дополнительного образования относится к классу задач прикладной программной инженерии и требует опоры на устоявшиеся методологические основы. В данном разделе рассматриваются основные подходы к организации процесса разработки, применимые архитектурные паттерны, а также стандарты, регулирующие качество, безопасность и

документирование программных систем в контексте разрабатываемого продукта.

1.3.1 Методологии жизненного цикла разработки ПО

Методологии жизненного цикла разработки ПО определяют порядок и логику выполнения этапов создания программной системы, что напрямую влияет на управляемость, качество и сроки реализации проекта. Различные методы — каскадная (Waterfall), спиральная (Spiral), инкрементная, итеративная, гибкая (Agile) — подходят под разные организационные и технологические условия.

Для разработки систем, ориентированных на конечного пользователя с нечётко формализованными требованиями на старте проекта, наиболее применимы гибкие методологии семейства Agile. В частности, Scrum предполагает итеративную разработку короткими циклами (спринтами), что позволяет последовательно наращивать функциональность системы и своевременно корректировать приоритеты по результатам обратной связи от заказчика.

Применительно к разрабатываемой системе данный подход целесообразен по следующим причинам: функциональные требования охватывают несколько независимых предметных областей (CRM, учебный контур, финансы, геймификация, аналитика), каждую из которых можно реализовать и протестировать в рамках отдельного итерационного цикла. Это снижает риск накопления нереализованных зависимостей и позволяет предъявлять промежуточные рабочие версии продукта для верификации.

1.3.2 Архитектурные паттерны и подходы

Многоуровневая клиент-серверная архитектура

Современные образовательные платформы строятся на основе многоуровневой клиент-серверной архитектуры: уровень представления (веб-интерфейс), уровень бизнес-логики (серверная часть) и уровень хранения

данных (реляционная СУБД). Такое разделение обеспечивает независимое масштабирование каждого уровня, упрощает замену компонентов и поддерживает параллельную разработку frontend и backend частей.

RESTful API как интеграционная шина

Взаимодействие между клиентскими приложениями и серверной частью, а также интеграция с внешними сервисами (Telegram Bot API, платёжные системы, почтовые шлюзы) реализуются через RESTful-интерфейс. Данный подход обеспечивает платформенную независимость, предсказуемость контрактов обмена данными и лёгкость тестирования взаимодействий.

Модульная архитектура

Для систем с разнородными функциональными блоками актуален выбор между монолитной модульной архитектурой и микросервисным подходом. На этапе первоначальной разработки и при ограниченных ресурсах команды рациональным является модульный монолит: каждый функциональный домен (CRM, расписание, финансы, аналитика) инкапсулирован в свой модуль с чёткими внутренними границами. Это упрощает последующий переход к микросервисам по мере роста нагрузки и числа филиалов.

1.3.3 Стандарты в области качества и разработки программного обеспечения

ГОСТ Р ИСО/МЭК 25010-2015

Данный стандарт определяет модель качества программного продукта, включающую характеристики функциональной пригодности, производительности, совместимости, удобства использования, надёжности, защищённости, сопровождаемости и переносимости. Требования разрабатываемой системы — одновременная работа не менее 500 пользователей, поддержка актуальных браузеров, интуитивный интерфейс для разных ролей — схожи с характеристиками качества, описанными в данном

стандарте, и могут использоваться как критерии при тестировании и приёме системы.

ГОСТ Р ИСО/МЭК 12207-2010

Стандарт регулирует процессы жизненного цикла программных систем: от формирования требований и проектирования до сопровождения и вывода из эксплуатации. Применение данного стандарта обеспечивает системность в документировании процессов разработки и позволяет структурировать работы в виде последовательности стандартизованных процессов: определения требований, проектирования архитектуры, разработки, интеграции, тестирования и сопровождения.

1.3.4 Стандарты и нормативная база в сфере защиты данных и информационной безопасности

Федеральный закон № 152-ФЗ «О персональных данных»

Разрабатываемая система обрабатывает персональные данные несовершеннолетних и их родителей. Это предъявляет особые требования: получение согласия на обработку, разграничение доступа по ролям, хранение данных на территории Российской Федерации, а также обязательное ведение журнала операций с персональными данными. Соответствие закону рассматривается как нефункциональное требование системы и должно закладываться на уровне архитектуры хранения, а не достигаться позднейшими доработками.

ГОСТ Р 57580.1-2017

Стандарт устанавливает требования к обеспечению операционной надёжности и безопасности в информационных системах. Применительно к разрабатываемому продукту это означает: использование защищённых протоколов передачи данных (HTTPS/TLS), хранение паролей в хешированном виде, ограничение доступа к административным функциям, а также защиту от типовых классов веб-уязвимостей (SQL-инъекции, межсайтовый скриптинг и другие).

1.3.5 Стандарты документирования

Техническая и пользовательская документация к разрабатываемой системе оформляется в соответствии с принципами ГОСТ серии 19 (документирование программных средств) и ГОСТ серии 34 (информационные технологии, автоматизированные системы). В частности, применяются подходы к составлению технического задания, описания программы, руководства пользователя, соответствующие установленным требованиям к структуре и содержанию эксплуатационных документов.

1.4 Моделирование архитектуры: UML, IDEF0 и CASE-среды

Для полноценного описания архитектуры разрабатываемой системы было выполнено визуальное моделирование с использованием нотаций UML, IDEF0, DFD и диаграммы развертывания. Такой подход позволяет описать систему с разных точек зрения: функциональной, структурной, поведенческой и инфраструктурной.

На рисунке 1 представлена контекстная IDEF0-диаграмма, отражающая входы, выходы, механизмы и управляющие воздействия для автоматизированной системы школы программирования.

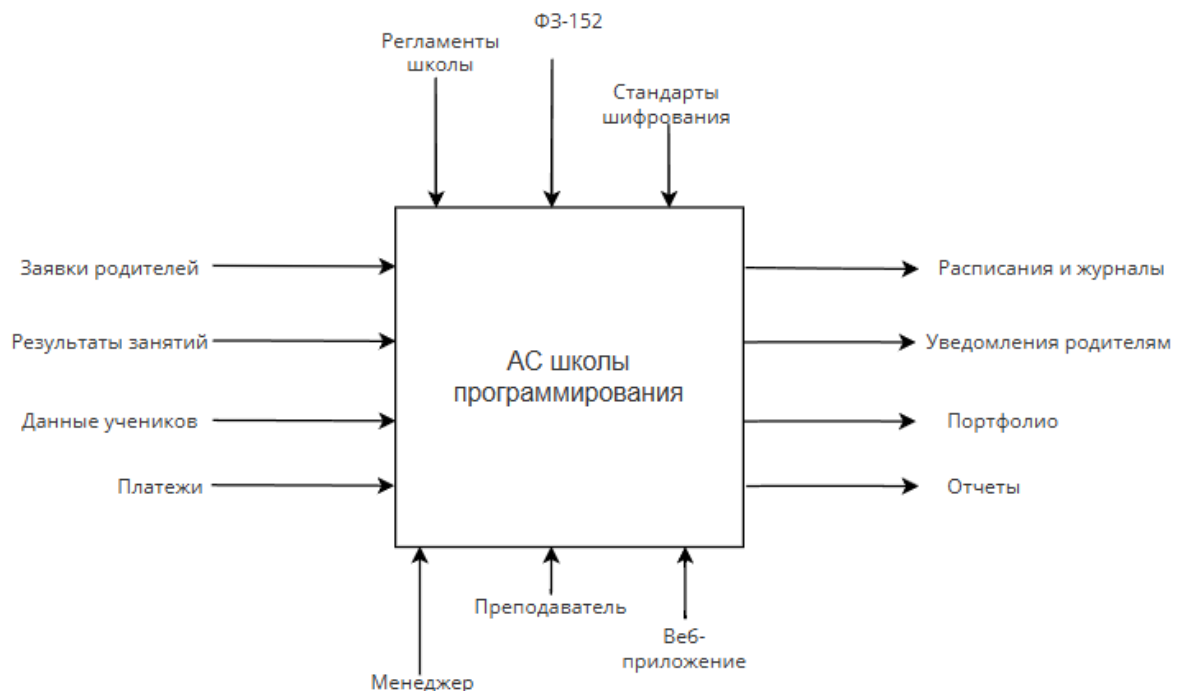


Рисунок 1 – Диаграмма IDEF0 верхнего уровня

На рисунке 2 представлена диаграмма вариантов использования, показывающая ключевых акторов системы: менеджера, преподавателя, родителя и ученика, а также их основные сценарии взаимодействия.



Рисунок 2 – Диаграмма вариантов использования

В связи с высокой сложностью предметной области разрабатываемой системы единая диаграмма классов была разделена на четыре самостоятельных модуля (рисунки 3-6): «Пользователи и роли», «Учебный процесс», «Финансы и уведомления», «Портфолио и геймификация».



Рисунок 3 – Диаграмма классов модуля «Пользователи и роли»

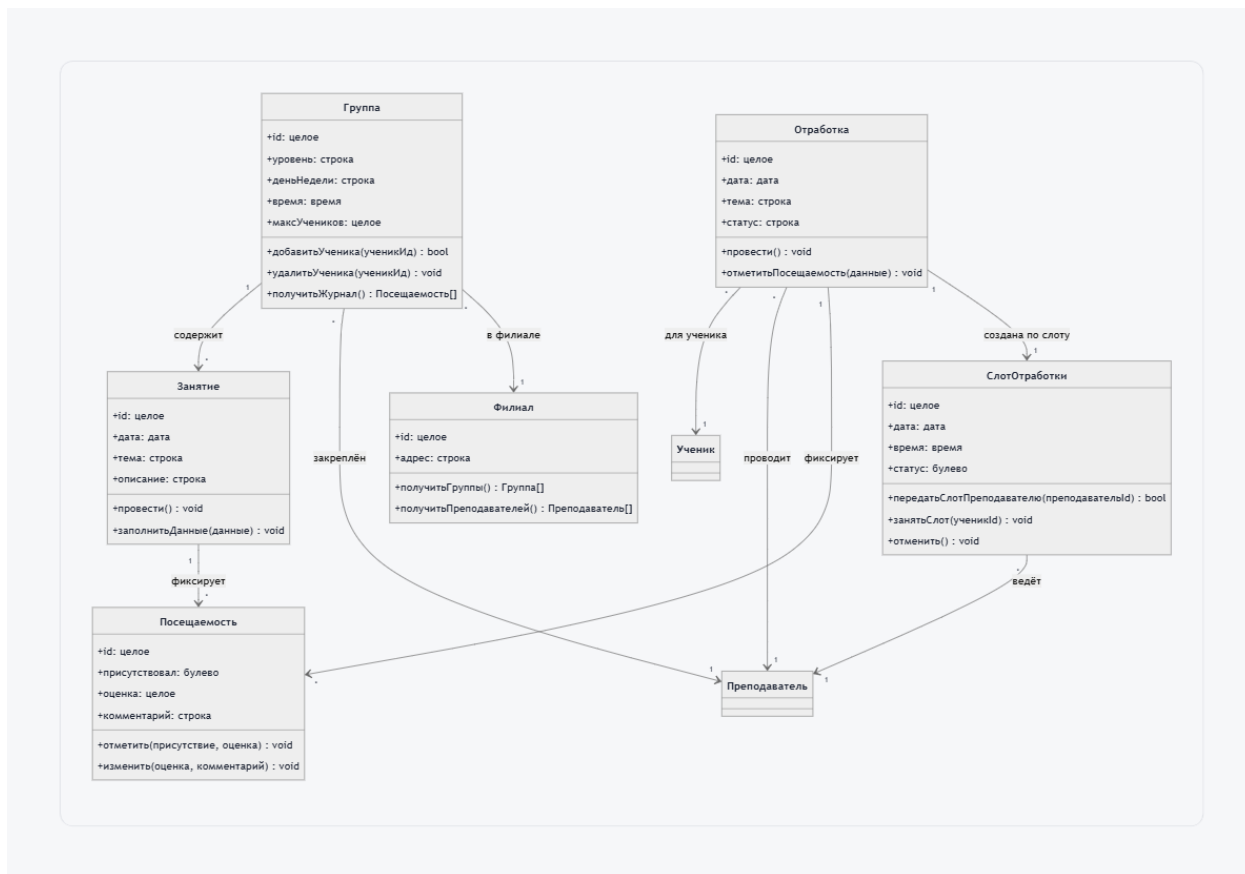


Рисунок 4 – Диаграмма классов модуля «Учебный процесс»

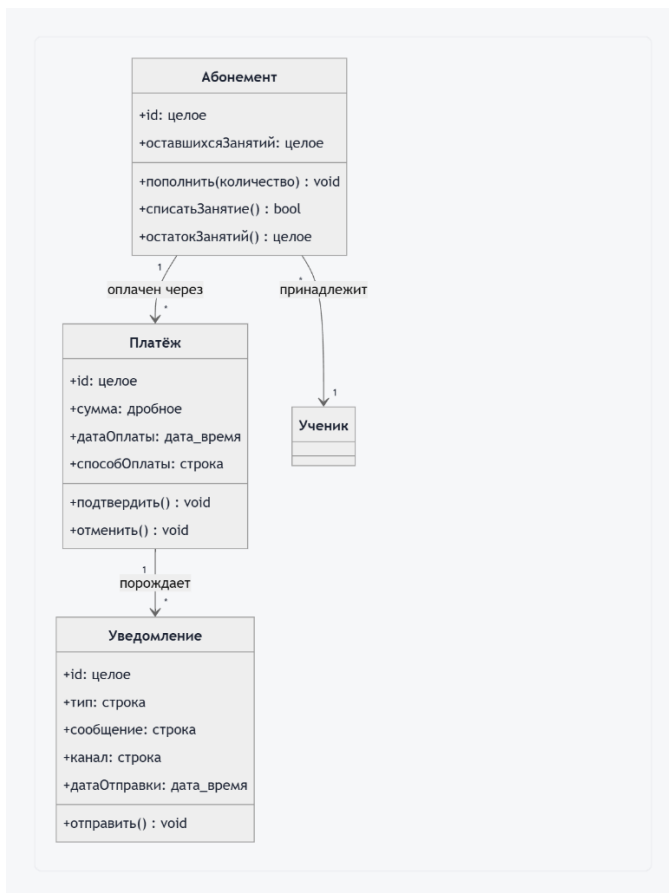


Рисунок 5 – Диаграмма классов модуля «Финансы и уведомления»



Рисунок 6 – Диаграмма классов модуля «Портфолио»

Поведенческие сценарии системы представлены UML-диаграммами последовательности и активности. Диаграмма последовательности смарт-отработки пропусков (рисунок 7) описывает недельный цикл публикации слотов преподавателем, выбор слота родителем и подтверждение через уведомления. Диаграмма активности антиотток-аналитики (рисунок 8) отражает логику автоматического формирования списка учеников в зоне риска.

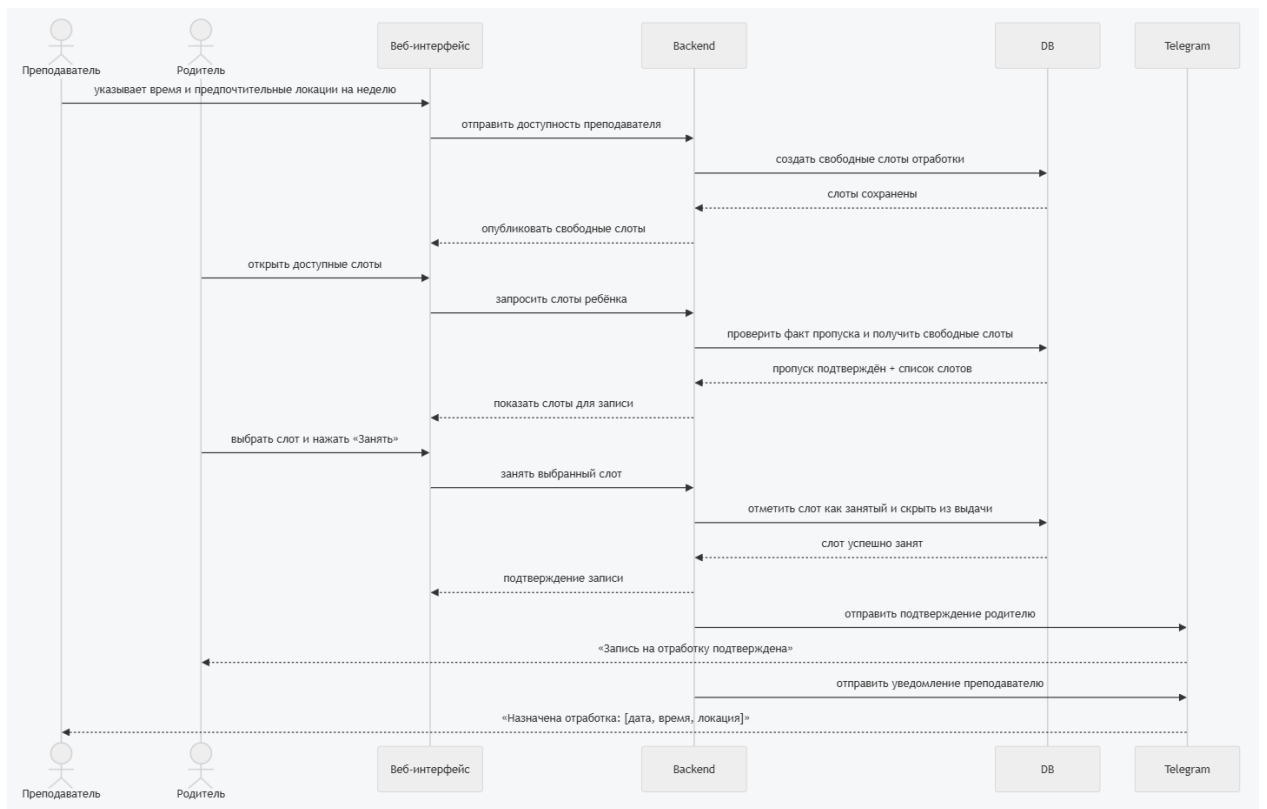


Рисунок 7 – Диаграмма последовательностей «Отработка пропуска»

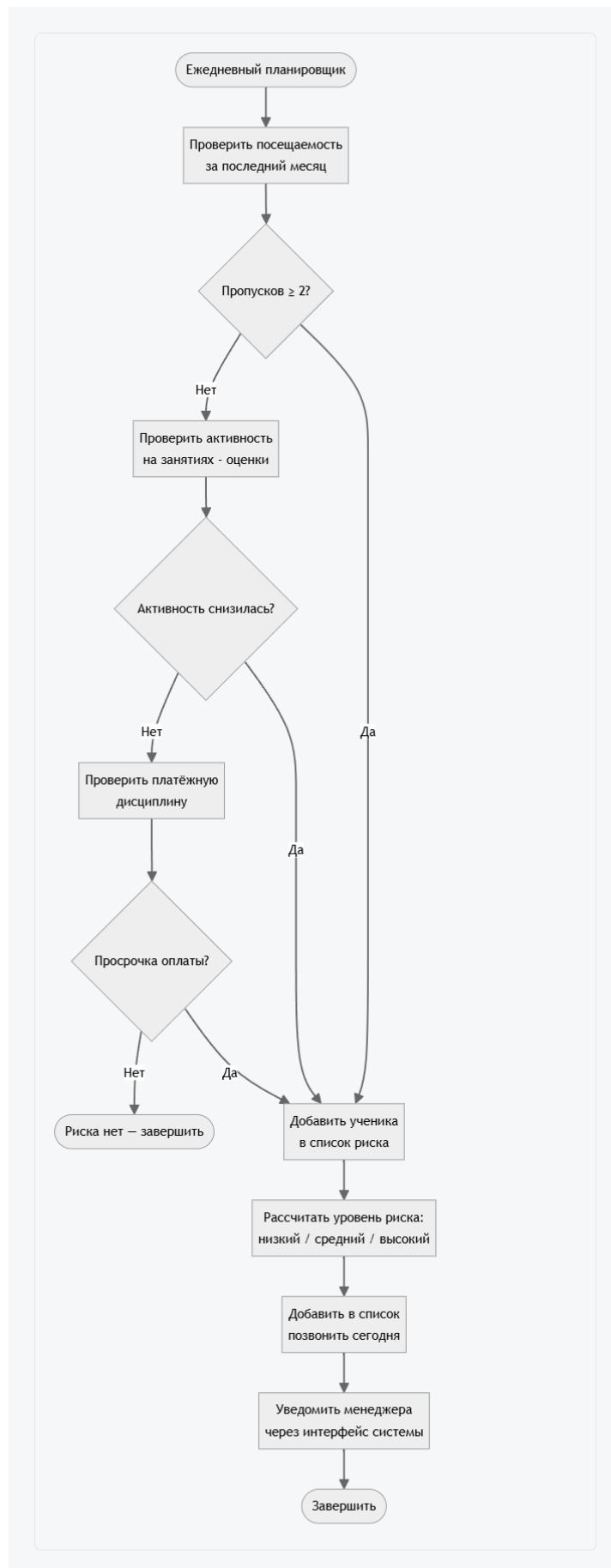


Рисунок 8 – Диаграмма деятельности «Антиотток-аналитика»

Для описания потоков данных использованы две DFD-диаграммы: учебный контур и финансы (рисунок 9), а также контур портфолио и недельной ленты проектов (рисунок 10). Это обеспечивает прозрачность движения данных между ролями, процессами и хранилищами.

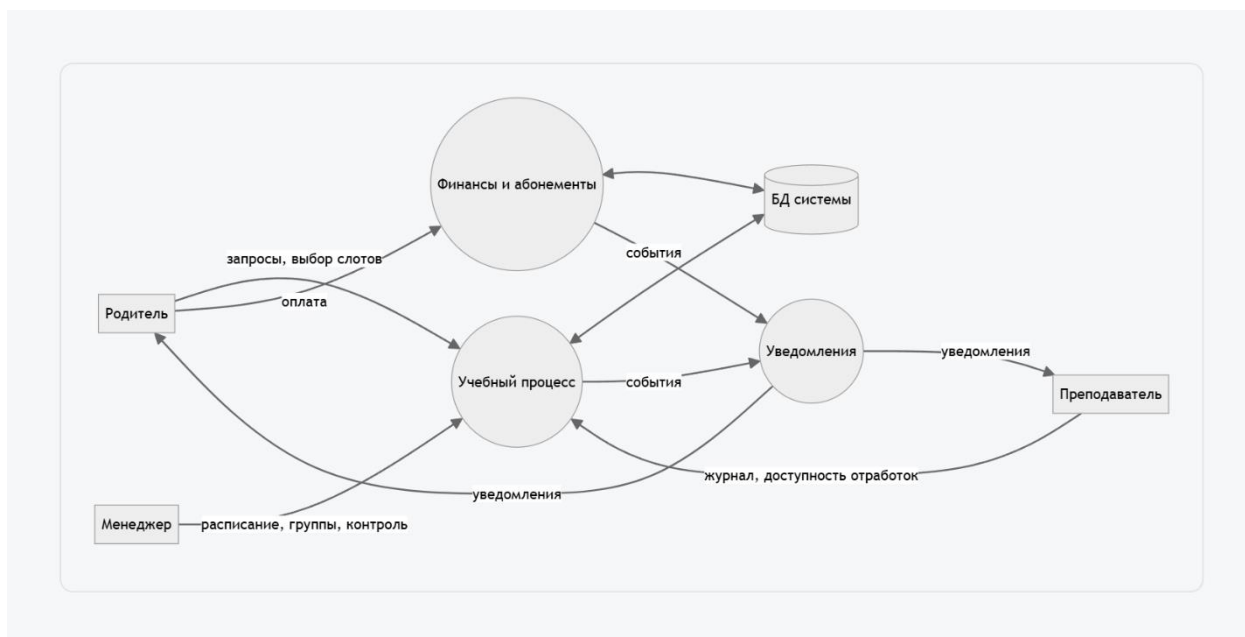


Рисунок 9 – DFD «Учебный процесс и финансы»

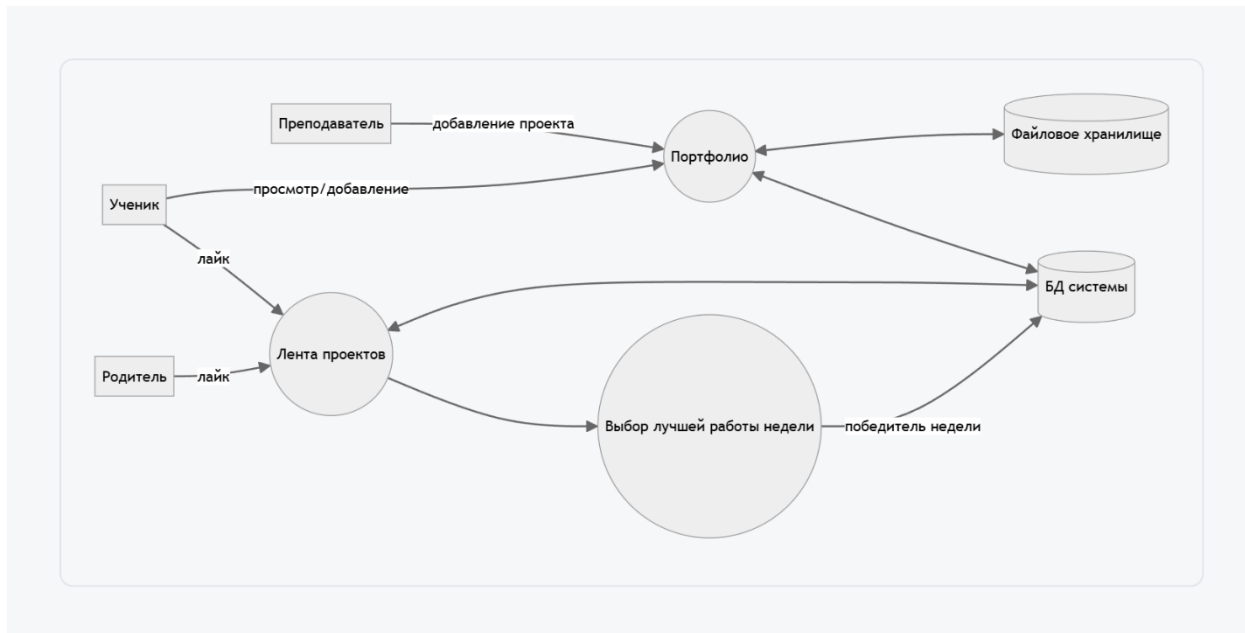


Рисунок 10 – DFD «Портфолио и лента проектов»

Инфраструктурный уровень системы представлен диаграммой развертывания (рисунок 11), где показаны браузерный клиент, серверные сервисы, база данных PostgreSQL, файловое хранилище и внешняя интеграция с Telegram Bot API.

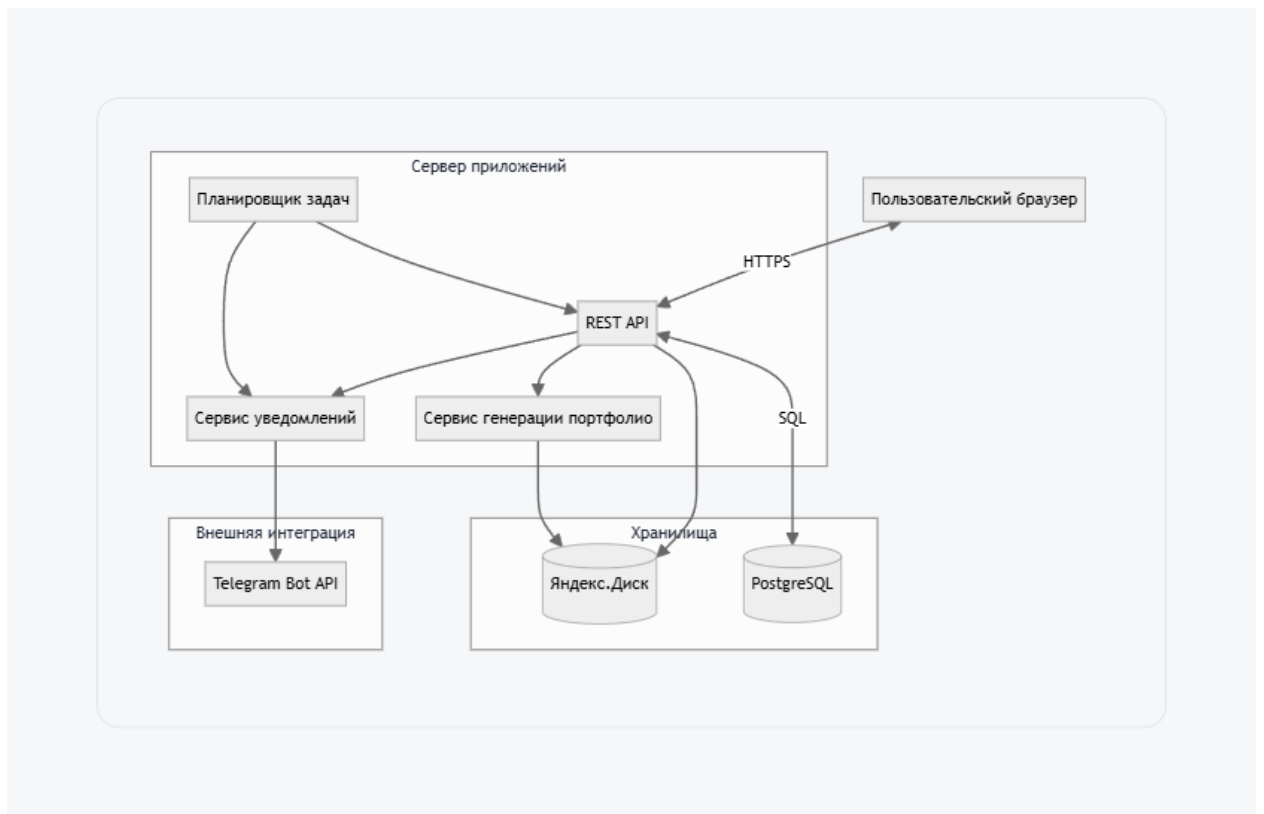


Рисунок 11 – Диаграмма развертывания системы

Глава 2. Проектирование и реализация автоматизированной системы для школы программирования

2.1 Архитектурное решение

Архитектура системы построена по многоуровневому клиент-серверному принципу. Клиентский уровень реализует веб-интерфейс для всех ролей пользователей. Серверный уровень включает REST API, планировщик фоновых задач, сервис уведомлений и сервис генерации PDF-портфолио. Уровень данных содержит реляционную БД и файловое хранилище. Такое разделение обеспечивает масштабируемость, управляемость и возможность поэтапного развития функциональности.

2.2 Ключевые функциональные модули

В рамках реализации выделены четыре основные функциональные области:

- модуль пользователей и ролей (аутентификация, разграничение доступа, привязка родителя к ученику);
- модуль учебного процесса (группы, расписание, журнал посещаемости, отработки);
- модуль финансов и уведомлений (абонементы, платежи, автоматические сообщения);
- модуль портфолио и проектной ленты (добавление работ, голосование, определение лучшей работы недели).

2.3 Выбор программных решений

2.3.1 CASE – средства

Для реализации системы были выбраны следующие CASE-средства:

- Visual Studio Code – текстовый редактор для работы с кодом;
- pgAdmin – создание и администрирование базы данных PostgreSQL;

- Docker Desktop – контейнеризация и локальное развертывание приложения;
- GitHub – хранение кода, контроль версий.

2.3.2 Стек серверной части

Для реализации серверной части был выбран следующий стек технологий:

- Python 3.10+ – язык для разработки логики приложения;
- Django + Django REST Framework – веб-фреймворк и REST API;
- PostgreSQL – реляционная база данных.

2.3.3 Стек клиентской части

Для реализации клиентской части был выбран следующий стек технологий:

- React – библиотека для пользовательского интерфейса;
- Fetch API – взаимодействие с серверным API;
- CSS – стилизация интерфейса;
- Chart.js – отображение статистики и графиков.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта сформирована и обоснована архитектура автоматизированной системы управления образовательными процессами для школы программирования. Проведен анализ существующих отечественных и зарубежных решений, сформулированы функциональные и нефункциональные требования, определены методологические и нормативные основания проектирования.

Разработанный комплект моделей описывает систему на уровнях процессов, структуры, взаимодействий и инфраструктуры. Это обеспечивает целостное представление о проектируемом решении и снижает риски при переходе к реализации.

Предложенная архитектура будет использована как основа для дальнейшего развития продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. PyCharm [Электронный ресурс]. URL: <https://www.jetbrains.com/pycharm/> (дата обращения 08.01.2026).
2. Docker Desktop [Электронный ресурс]. URL: <https://www.docker.com/products/docker-desktop/> (дата обращения 08.01.2026).
3. GitHub [Электронный ресурс]. URL: <https://github.com/> (дата обращения 08.01.2026).
4. PgAdmin [Электронный ресурс]. URL: <https://www.pgadmin.org/> (дата обращения 08.01.2026).
5. Django documentation [Электронный ресурс]. URL: <https://docs.djangoproject.com/en/6.0/> (дата обращения 08.01.2026).
6. Django REST Framework documentation [Электронный ресурс]. URL: <https://www.django-rest-framework.org/> (дата обращения 08.01.2026).
7. Telegram Bot API Documentation [Электронный ресурс]. URL: <https://core.telegram.org/bots/api>